Vibe Coding prompt

Act as a Solidity and Python Web3 developer. Build compile and review a supplier purchase order for an isolated test chain. Use Solidity 0.8.30 and web3.py 8.0.0 with EthereumTesterProvider backed by Py-EVM; also allow --rpc for an isolated unlocked local JSON-RPC test chain. Requirements: enum state, immutable quantity and supplier-only transitions implement a business requirement trace. Read mock-data.json, derive a canonical SHA-256 digest where required, compile using pinned solc-js 0.8.30, deploy actual EVM bytecode, wait for every receipt and assert receipt status. Include positive and negative authorization/state tests, output chain ID/address/transaction hashes/gas and a deployment manifest. Do not simulate contract state in Python or use mainnet keys. Use OpenZeppelin 5.4.0 for ERC standards. Return Contract.sol, run_activity.py, mock-data.json, compile.cjs, requirements.txt and a detailed troubleshooting guide. Explain integer units, failure cases and what the assertions prove. Never claim a browser wallet or public testnet succeeded unless actually exercised.

Review prompt

Review every external function for authorization, state transitions, integer units and invariants. Point to exact lines. Produce a requirement-to-assertion matrix and add a failing test before repairing any defect.

Scenario-specific generation constraints

Read quantity from mock data; constructor stores supplier account1, quantity and hashed record ID. Attempt accept as account2, then accept as supplier. Read state=1. Deliver as supplier, read state=2 and reject repeated deliver.

Expected assertions: quantity=25; state Created0→Accepted1→Delivered2; outsider/repeated calls fail.

Extension prompt

Add buyer confirmation as state3 with buyer-only transition and test supplier cannot confirm. Return the exact changed source lines and positive/negative tests.